FULL STACK COURSE ENROLLMENT SYSTEM – EDU ENROLL

A Full Stack Development Lab Project Report *Submitted in partial fulfillment of the requirements for the award of the degree of* **Bachelor of Technology** *in* **Computer Science & Engineering.**

Submitted By

K.S.T. Ramya Sri	25A35A0535	CSE-F
N. Param Singh Naik	25A35A0539	CSE-F
K. Reshma Srivalli	24A31A05JH	CSE-F
N. Sampath	24A31A05LB	CSE-F

Department of Computer Science & Engineering

PRAGATI ENGINEERING COLLEGE

(Approved by AICTE, Permanently Affiliated to JNTUK, Kakinada)

Surampalem, Andhra Pradesh Academic Year: 2025 – 2026

TABLE OF CONTENTS

Abstract	3
Introduction	3
Objectives	4
Technologies Used	5
System Architecture	6
Methodology/ Working	7-8
Module Description	8
Database Design	9
Implementation	10
Output Screens	11 - 12
Future Scope	13
Conclusion	13

ABSTRACT

EduEnroll is a production-grade course enrollment platform developed as a Full Stack Development Lab project at Pragati Engineering College. While traditional platforms suffer from data corruption during course updates, EduEnroll introduces a **Delta-Patching Algorithm** to preserve student progress seamlessly. The system follows a **95+5 Progress Rule**, gating professional certification behind a mandatory quiz to ensure learning integrity. Built using the **React-Flask-PostgreSQL (PERN/RFP)** stack, it features a 3D gamification system with learning streaks, automated PDF certificate generation, and an SMTP-integrated notification system. The result is a robust, scalable ecosystem that bridges the gap between passive video watching and active skill mastery.

INTRODUCTION

EduEnroll is a production-grade, full-stack course enrollment platform engineered to transform the traditional e-learning experience into an **intelligent learning ecosystem**. Developed as a Major Lab Project for the Full Stack Development curriculum, this system moves beyond simple video hosting to provide a robust environment focused on data integrity, learner engagement, and verified skill acquisition.

In the contemporary educational landscape, digital platforms often face the "Progress Drift" dilemma- where structural changes to a course (such as adding or removing lessons) lead to corrupted student progress data. **EduEnroll** addresses this technical challenge head-on, ensuring a seamless and reliable journey for the modern learner.

Real-World Context

Standard Learning Management Systems (LMS) often suffer from a lack of genuine validation. Many students receive certifications based solely on "watch time" rather than actual comprehension. Additionally, from a technical standpoint, maintaining a consistent state between a dynamic frontend and a relational database during high-frequency updates remains a challenge for many entry-level systems.

The EduEnroll Innovation

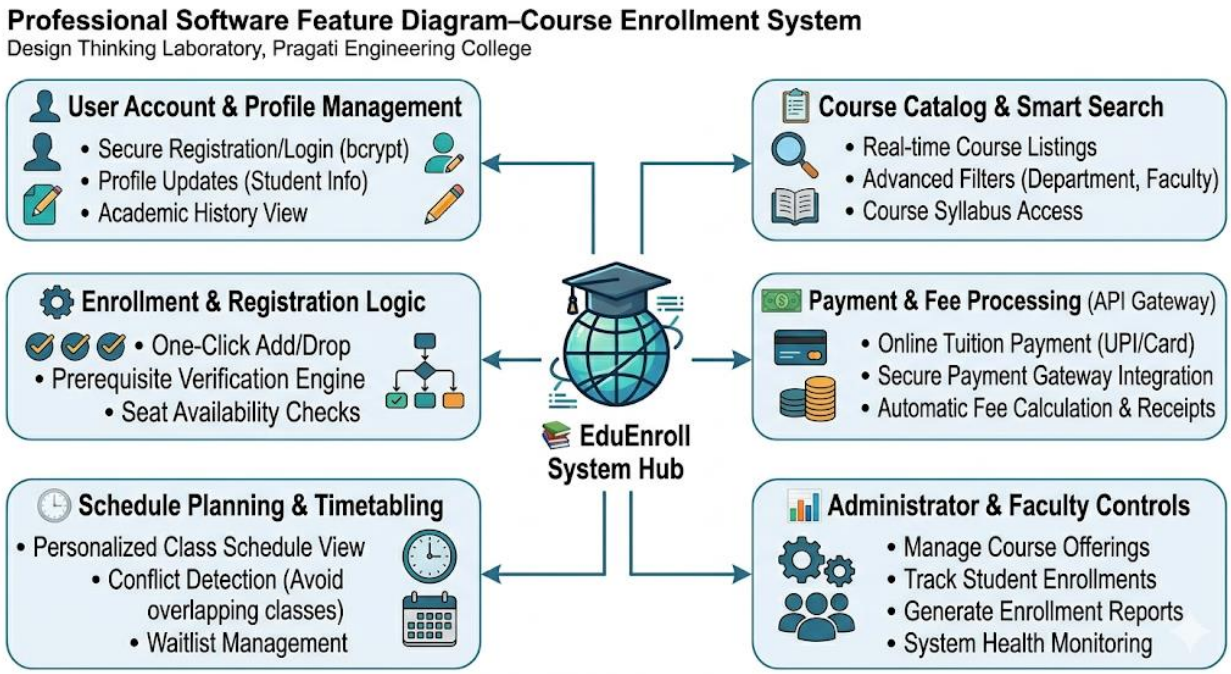
To solve these real-world issues, our platform implements two core proprietary mechanisms:

- **Delta-Patching Algorithm:** A sophisticated backend logic that diffs new course versions against old ones, preserving student progress states even when instructors modify course content.
- **The 95 + 5 Progress Rule:** A pedagogical framework that gates the final 5% of course completion—and the subsequent certification—behind a mandatory comprehension quiz.

OBJECTIVES

The primary goal of **EduEnroll** is to solve the technical and pedagogical flaws found in standard learning platforms. The project is designed with the following core objectives:

- **Implement Smart Progress Synchronization:** To develop and integrate a **Delta-Patching Algorithm** that prevents "Progress Drift," ensuring student completion data remains accurate even when instructors modify course content.
- **Enforce Knowledge Validation:** To move beyond passive consumption by implementing the **95 + 5 Progress Rule**, where the final 5% of progress is strictly gated behind a mandatory assessment.
- **Enhance Student Retention via Gamification:** To build a high-engagement ecosystem using a **3D Badge System** and **Learning Streak Trackers** to incentivize consistent study habits.
- **Automate Administrative Workflows:** To eliminate manual overhead by automating the generation and email delivery of **PDF Certificates** using server-side logic (FPDF & Flask-Mail).
- **Provide Role-Based Access Control (RBAC):** To architect three distinct portals (Admin, Instructor, Student) with secure, stateless authentication via **JSON Web Tokens (JWT)**.



TECHNOLOGIES USED

The system is built using a modern **RFP Stack (React, Flask, PostgreSQL)**, ensuring a clear separation between the presentation layer and business logic.

Frontend (Presentation Layer)

- **React 18:** For building a dynamic, component-based Single Page Application (SPA).
- **Vite:** Used as the next-generation build tool for lightning-fast development and optimized production bundles.
- **Tailwind CSS:** For utility-first responsive styling and rapid UI development.
- **Framer Motion:** To handle smooth transitions and premium 3D animations for the gamification system.
- **Lucide React:** For a clean, consistent, and lightweight icon library.

Backend (Application Layer)

- **Flask:** A lightweight Python micro-framework used to build the **RESTful API**.
- **SQLAlchemy (ORM):** To interact with the database using Python objects instead of raw SQL, ensuring better security and maintainability.
- **Flask-JWT-Extended:** For handling secure login sessions and role-based permissions through tokens.
- **FPDF:** For real-time, server-side generation of personalized PDF certificates.
- **Flask-Mail:** To handle SMTP integration for sending automated emails and certifications.

Database (Data Layer)

- **PostgreSQL:** A powerful, open-source relational database used to store complex relationships between users, courses, lessons, and badges.
- **Alembic (Flask-Migrate):** For handling database migrations and version control of the schema.

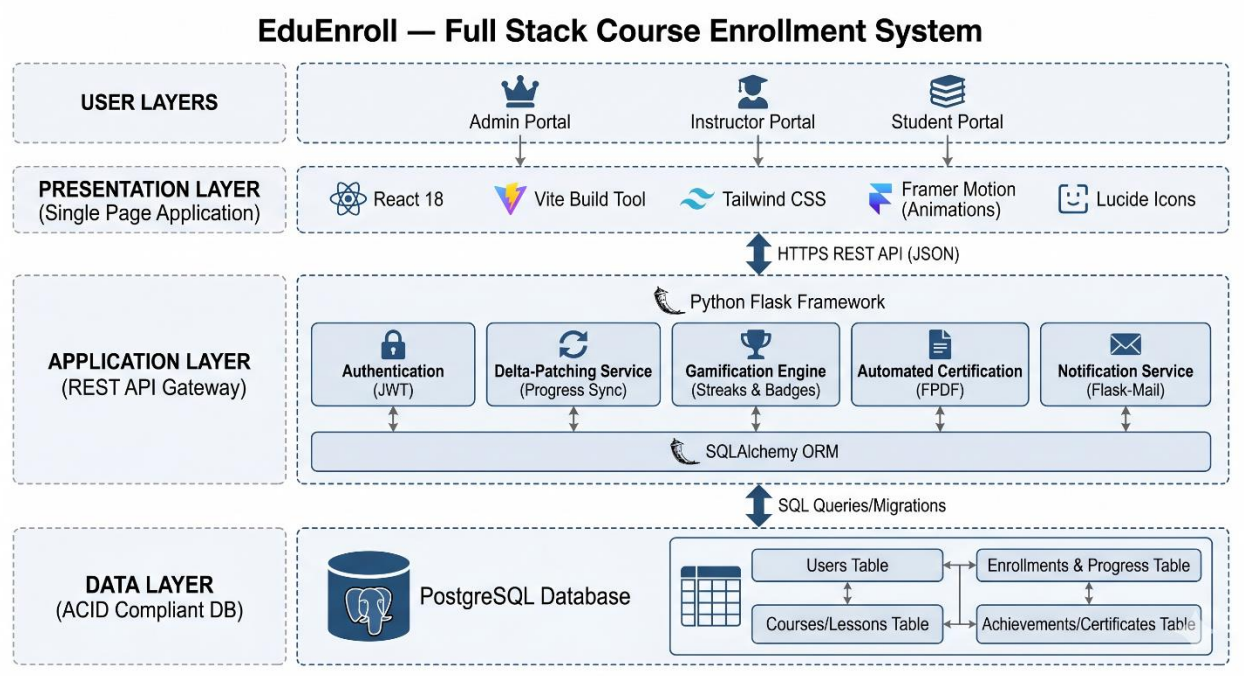
Development Tools

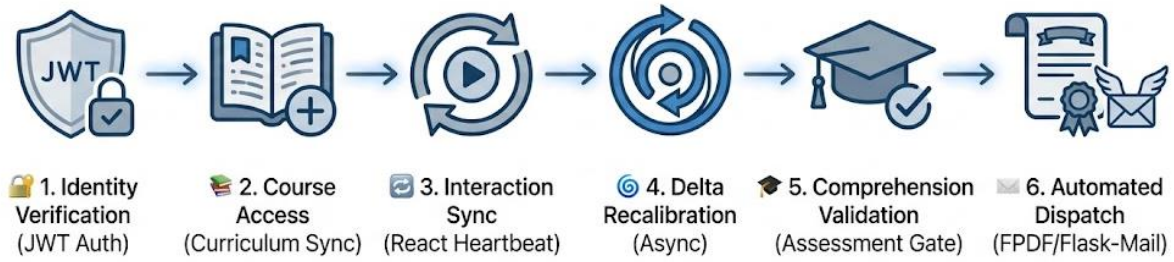
- **Git & GitHub:** For version control and team collaboration.
- **Postman:** For testing and documenting API endpoints.
- **VS Code:** Primary IDE for development.

SYSTEM ARCHITECTURE

The platform operates on a **Decoupled Architecture**:

1. **Client-Side:** The React frontend makes asynchronous HTTP requests to the backend.
2. **Server-Side:** The Flask API validates the JWT, processes business logic (like the Delta-Patching), and communicates with the DB.
3. **Database:** PostgreSQL maintains ACID compliance for all enrollment and progress transactions.





METHODOLOGY / WORKING

The development and operational logic of **EduEnroll** are rooted in modern software engineering practices, ensuring that the transition from raw data to a verified certificate is both secure and seamless.

Development Methodology

We adopted an **Agile-Iterative Framework** to manage the project lifecycle. This allowed us to build the system in modular phases, ensuring each layer was stable before moving to the next:

1. **System Design:** Formulating the relational schema in PostgreSQL and designing the RESTful API endpoints.
2. **API Development:** Constructing the Flask backend to handle core business logic, such as the Delta-Patching algorithm and JWT authentication.
3. **Frontend Engineering:** Building the React SPA, focusing on state management to ensure real-time UI updates for progress and badges.
4. **Integration & QA:** Connecting the services and performing stress tests on the certification engine to ensure email delivery was consistent.

Functional Workflow (System Logic)

The "Working" of the platform is defined by a continuous feedback loop between the client and the server. Below is the technical execution of a standard user session:

1. Stateless Authentication & RBAC

When a user logs in, the server issues a **JSON Web Token (JWT)**. This token contains the user's identity and role (Student, Instructor, or Admin). The frontend stores this token and includes it in the header of every API request, allowing the server to enforce **Role-Based Access Control (RBAC)** without maintaining a constant session on the server.

2. The Smart Progress Engine

As students interact with the course material, the system tracks progress through a **two-phase verification process**:

- **The Learning Phase:** The system monitors lesson completion. If an instructor updates the course structure (adding/removing modules), the **Delta-Patching Algorithm** recalculates the total weight of the course while preserving the student's previously completed "checked" states.
- **The Assessment Phase:** Once all instructional content is consumed, the system triggers the **Assessment Gate**. This unlocks the final exam, shifting the logic from passive tracking to active evaluation.

3. Automated Certification Pipeline

Upon a successful passing grade in the final quiz, the backend initiates a three-step automation sequence:

1. **Verification:** The server confirms that 100% of the criteria (Lessons + Quiz) have been met.
2. **Generation:** The **FPDF library** dynamically pulls the student's name, the course title, and the completion date to generate a unique PDF document.
3. **Dispatch:** The **Flask-Mail service** establishes an SMTP connection and dispatches the certificate directly to the student's inbox, while simultaneously updating the Achievement Gallery in the Student Portal.

4. Gamification Logic

The **Gamification Engine** runs asynchronously. It monitors database triggers—such as login frequency and completion rates—to update "Learning Streaks" and award 3D badges. This adds a psychological layer of motivation, incentivizing users to return to the platform daily.

MODULES DESCRIPTION

The system is architected into three distinct functional modules, each tailored to specific user roles and administrative requirements:

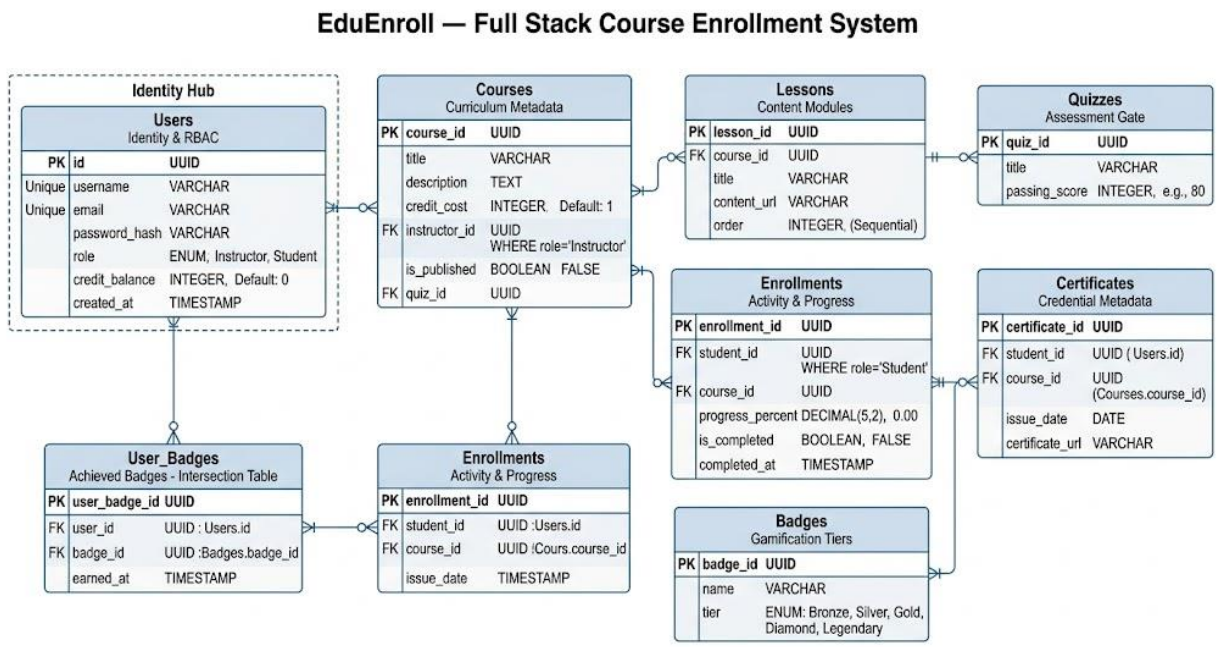
- **Administrative Oversight Module:** This is the command center of the platform. It provides high-level control over the user base, including instructor approval workflows and platform-wide analytics. A key feature here is the **Credit Management System**, which allows administrators to distribute learning credits to students, facilitating a controlled enrollment environment.
- **Instructional Design Module:** Designed for subject matter experts, this module provides the tools necessary to build comprehensive digital curricula. Instructors can manage lesson hierarchies, upload video content, and design assessment rubrics. It also features a real-time monitoring dashboard to track aggregate student performance across all hosted courses.
- **Student Engagement & Learning Module:** This is the primary interface for the end-user. It focuses on a frictionless learning experience, featuring a dynamic course browser, a granular progress dashboard, and an integrated assessment portal. The module also houses the **Personalized Achievement Gallery**, where earned badges and certifications are displayed.

DATABASE DESIGN

The data layer is managed by **PostgreSQL**, utilizing a relational schema designed for high consistency and referential integrity.

Key Entity Relationships

- **Users & Roles:** A centralized authentication table handles identity management for Students, Instructors, and Admins via role-based flags.
- **Course-Lesson Hierarchy:** A one-to-many relationship exists between courses and their constituent lessons, allowing for complex, multi-modular course structures.
- **Enrollment & Progress Tracking:** This intersection table maps students to courses and maintains a live state of lesson completion, quiz scores, and the timestamps for the **Learning Streak** logic.
- **Achievement Mapping:** Links users to specific badge IDs and certificate metadata, ensuring that once a credential is earned, it is permanently and securely logged.



IMPLEMENTATION

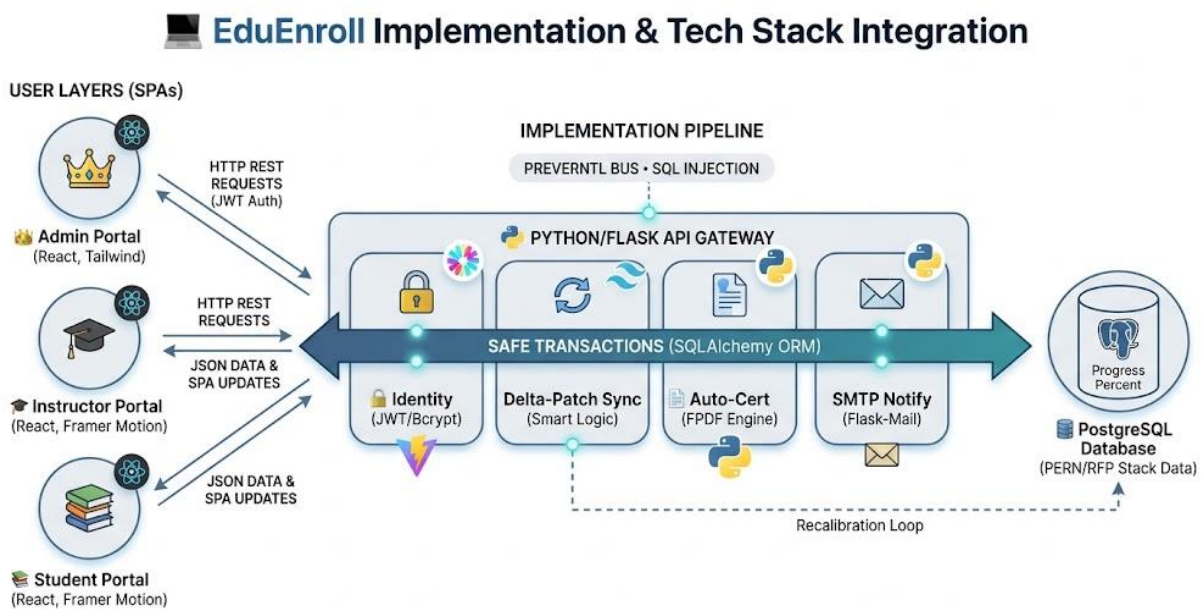
The implementation phase focused on the strict separation of concerns through a **RESTful API architecture**.

Backend Logic (Python/Flask)

The server-side implementation prioritizes security and automation. We utilized **SQLAlchemy ORM** to handle database transactions safely, preventing SQL injection. The core innovation lies in the **Delta-Patching Service**, which runs on the backend to reconcile course structural changes with existing student records. Additionally, the **Automated Certification Engine** utilizes the **FPDF library** to draw student-specific data onto a template, which is then dispatched via an **SMTP-configured Flask-Mail service**.

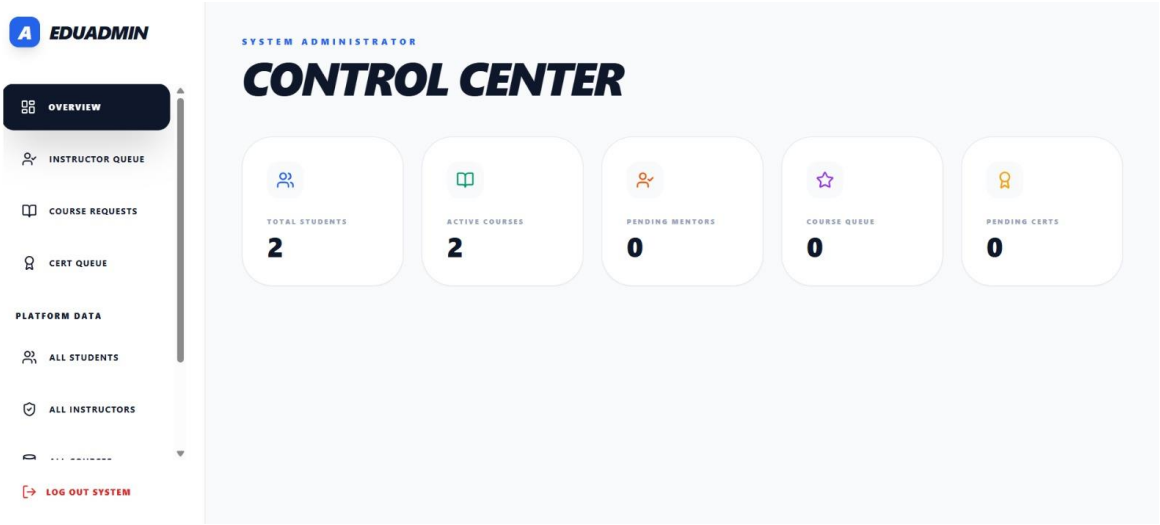
Frontend Interface (React/Vite)

The UI is built as a **Single Page Application (SPA)**, ensuring that transitions between lessons and portals are instantaneous. **Tailwind CSS** provides a modern, responsive design system, while **Framer Motion** powers the interactive 3D badge animations. State management is handled through React hooks, ensuring that the progress bars and achievement notifications update in real-time without requiring a page refresh.

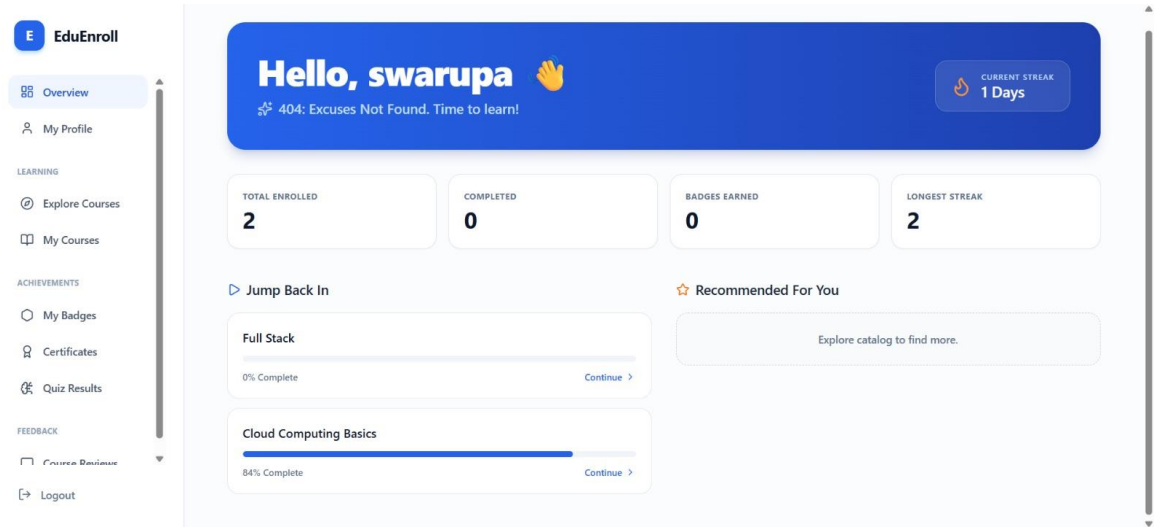


OUTPUT SCREENS

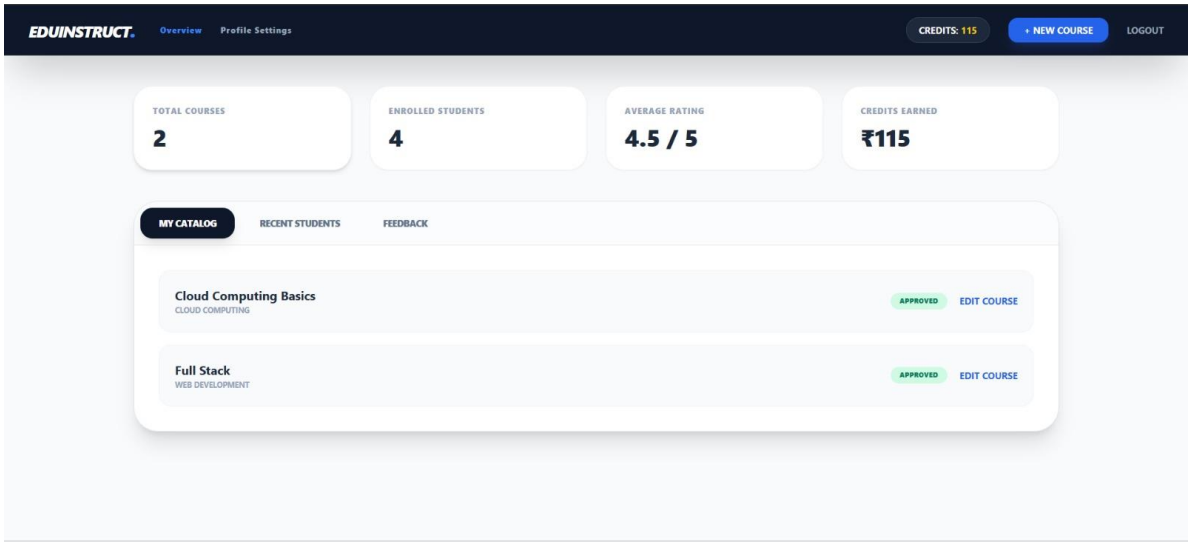
1. Admin Portal



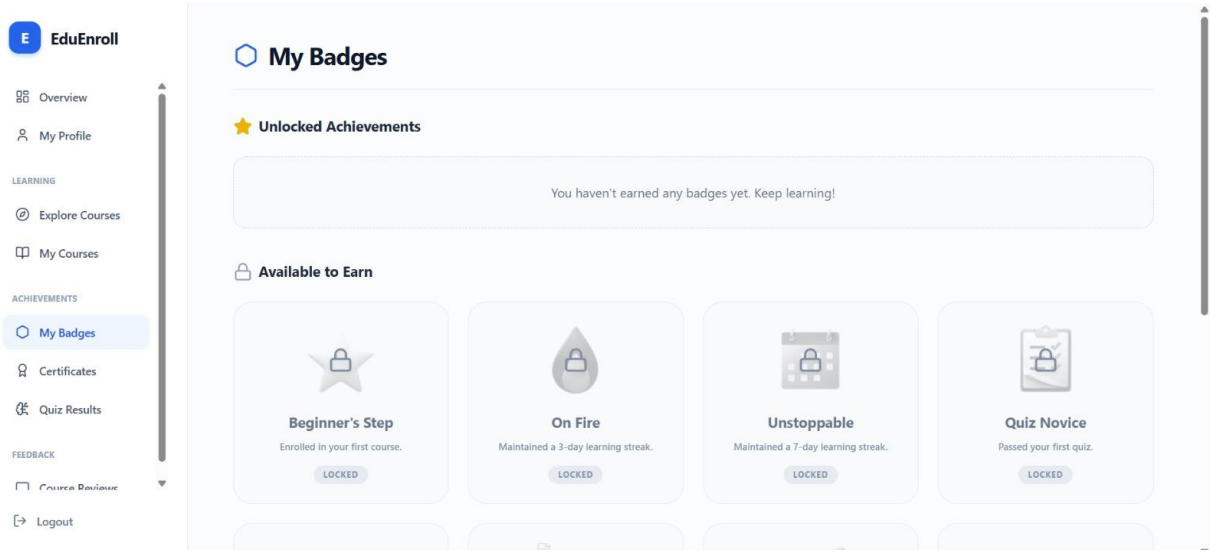
2. Student Portal



3. Instructor Portal



4. Gamification System



FUTURE SCOPE

While the current version of **EduEnroll** is a fully functional production-grade platform, there are several avenues for future enhancement to keep pace with evolving educational technologies:

- **AI-Powered Personalization:** Integration of Machine Learning models to analyze student performance and provide personalized course recommendations or "smart study" schedules.
- **Mobile Application Development:** Extending the ecosystem to native iOS and Android applications using **React Native**, enabling offline learning and push notifications for streak reminders.
- **Live Interactive Classrooms:** Integration of WebRTC or third-party API support (like Zoom or Daily.co) to allow instructors to host live sessions directly within the portal.
- **Blockchain-Verified Certificates:** Migrating the certification logic to a blockchain ledger (e.g., Polygon or Ethereum) to provide immutable, tamper-proof credentials that employers can verify instantly.
- **Advanced Analytics for Instructors:** Developing a deeper "Heatmap" analytics dashboard for instructors to see exactly where students drop off in a video or struggle in a quiz.
- **Collaborative Learning Hubs:** Adding peer-to-peer discussion forums and collaborative project workspaces to foster a community-based learning environment.

CONCLUSION

EduEnroll successfully demonstrates the potential of a modern, full-stack approach to digital education. By moving away from static learning management and adopting an **intelligent ecosystem** model, the project addresses critical industry challenges such as data corruption during content updates and the devaluation of digital credentials.

The implementation of the **Delta-Patching Algorithm** ensures a high level of technical reliability, while the integration of **gamification** and **automated certification** creates a compelling, user-centric experience. Developed within the academic framework of **Pragati Engineering College**, this platform serves as a robust blueprint for scalable, secure, and engaging educational software, proving that thoughtful backend logic can significantly enhance the integrity of the learning journey.

THANK YOU